

Arithmetic Logic Unit (ALU)

Project Description Report

Name: Abdelrahman Hany Elatroush	ID: 231001741
Name: Ahmed Ashraf Mokhtar	ID: 231001773
Name: Ahmed Hassan Shata	ID: 231000267
Name: Ahmed Mohamed Nour	ID: 23100 2375
Name: Moaz Khaled Ibrahim	ID: 231001564

Table of Contents

1. Verilog Architecture and Module Hierarchy	3
1.1 Architecture Overview	3
1.2 Module Hierarchy Implementation	4
1.2.1 Mux21	4
1.2.2 Mux41	4
1.2.3 full_adder.....	5
1.2.4 AddSub (operation 0 - 1)	6
1.2.5 multiplier (operation 2)	7
1.2.6 block1_mod	8
1.2.7 sign_mux.....	8
1.2.8 Reminder_b.....	9
1.2.9 Modulus (operation 3).....	10
1.2.10 selectors	11
1.2.11 shifting_circuit.....	11
1.2.12 ASR (operation 4).....	12
1.2.13 ALU_top (Top module for ALU).....	13
2. ModelSim Waveform Verification	14
2.1 Simulation Setup	14
2.2 Files used	15
2.2.1 Golden Model.....	15
2.2.2 Testbench	16
2.2.3 Source list.....	17
2.2.4 Do file	18
2.3 Simulation log and waveforms.....	19
3. Conclusion	19
4. Appendix.....	20

1. Verilog Architecture and Module Hierarchy

1.1 Architecture Overview

The designed Arithmetic Logic Unit (ALU) operates on two 4-bit signed inputs, denoted as A and B, encoded using two's complement representation. The operation is selected using a 3-bit control signal (Sel).

The ALU produces an 8-bit signed output, which is required to correctly represent the result of signed multiplication.

In addition to the main output, the ALU generates three status flags:

- **Sign_flag:** Indicates whether the result is negative. It is directly taken from the most significant bit (MSB) of the output.
- **Zero_Flag:** Asserted when the output value equals zero.
- **DZ_Flag (Divide-by-Zero Flag):** Asserted only when a modulus operation is requested and the divisor B = 0.

All operations are evaluated in parallel, and the final result is selected using multiplexers controlled by the Sel signal.

A behavioral reference model (ALU_Gm) is used as a *golden model* for verification, while the actual design under test (ALU_top) is fully gate structurally.

1.2 Module Hierarchy Implementation

1.2.1 Mux21

```
1 module mux21(  
2     input A, B, S,  
3     output out  
4 );  
5     wire w1, w2, w3;  
6     assign w1 = ~S;  
7     assign w2 = A & w1;  
8     assign w3 = B & S;  
9     assign out = w2 | w3;  
10 endmodule
```

1.2.2 Mux41

```
1 module mux41 (  
2     input A0, A1, A2, A3, S0, S1,  
3     output out  
4 );  
5     wire w1, w2;  
6     mux21 inst1 (.A(A0), .B(A1), .S(S0), .out(w1));  
7     mux21 inst2 (.A(A2), .B(A3), .S(S0), .out(w2));  
8     mux21 inst3 (.A(w1), .B(w2), .S(S1), .out(out));  
9 endmodule
```

1.2.3 full_adder



```
1 module full_adder(  
2     input A, B, Cin,  
3     output out, cout  
4 );  
5     wire w1, w2, w3;  
6     assign w1 = A ^ B;  
7     assign out = w1 ^ Cin;  
8     assign w2 = w1 & Cin;  
9     assign w3 = A & B;  
10    assign cout = w2 | w3;  
11  
12 endmodule
```

1.2.4 AddSub (operation 0 - 1)

```
1 module AddSub(  
2     input A0, A1, A2, A3,  
3     input B0, B1, B2, B3,  
4     input Cin,  
5     output out0, out1, out2, out3, out4, out5, out6, out7  
6 );  
7  
8     wire w1, w2, w3, w4, w5, w6, w7, w8;  
9     wire w9, w10, w11, w12, w13, w14, w15, w16;  
10  
11     assign w1 = B0 ^ Cin;  
12     assign w2 = B1 ^ Cin;  
13     assign w3 = B2 ^ Cin;  
14     assign w4 = B3 ^ Cin;  
15     assign w5 = B3 ^ Cin;  
16     assign w6 = B3 ^ Cin;  
17     assign w7 = B3 ^ Cin;  
18     assign w8 = B3 ^ Cin;  
19  
20     full_adder inst1 (.A(A0), .B(w1), .Cin(Cin), .out(out0), .cout(w9));  
21     full_adder inst2 (.A(A1), .B(w2), .Cin(w9), .out(out1), .cout(w10));  
22     full_adder inst3 (.A(A2), .B(w3), .Cin(w10), .out(out2), .cout(w11));  
23     full_adder inst4 (.A(A3), .B(w4), .Cin(w11), .out(out3), .cout(w12));  
24     full_adder inst5 (.A(A3), .B(w5), .Cin(w12), .out(out4), .cout(w13));  
25     full_adder inst6 (.A(A3), .B(w6), .Cin(w13), .out(out5), .cout(w14));  
26     full_adder inst7 (.A(A3), .B(w7), .Cin(w14), .out(out6), .cout(w15));  
27     full_adder inst8 (.A(A3), .B(w8), .Cin(w15), .out(out7), .cout(w16));  
28  
29 endmodule
```

1.2.5 multiplier (operation 2)

```
1 module multiplier (  
2     input A0, A1, A2, A3,  
3     input B0, B1, B2, B3,  
4     output out0, out1, out2, out3, out4, out5, out6, out7  
5 );  
6  
7     wire pp00, pp01, pp02, pp03;  
8     wire pp10, pp11, pp12, pp13;  
9     wire pp20, pp21, pp22, pp23;  
10    wire pp30, pp31, pp32, pp33;  
11    wire ppn03, ppn13, ppn23, ppn30, ppn31, ppn32;  
12  
13    assign gnd = 0;  
14  
15    assign pp00 = A0 & B0;  
16    assign pp01 = A1 & B0;  
17    assign pp02 = A2 & B0;  
18    assign pp03 = A3 & B0;  
19    assign ppn03 = ~pp03;  
20  
21    assign pp10 = A0 & B1;  
22    assign pp11 = A1 & B1;  
23    assign pp12 = A2 & B1;  
24    assign pp13 = A3 & B1;  
25    assign ppn13 = ~pp13;  
26  
27    assign pp20 = A0 & B2;  
28    assign pp21 = A1 & B2;  
29    assign pp22 = A2 & B2;  
30    assign pp23 = A3 & B2;  
31    assign ppn23 = ~pp23;  
32  
33    assign pp30 = A0 & B3;  
34    assign ppn30 = ~pp30;  
35    assign pp31 = A1 & B3;  
36    assign ppn31 = ~pp31;  
37    assign pp32 = A2 & B3;  
38    assign ppn32 = ~pp32;  
39    assign pp33 = A3 & B3;  
40  
41    wire s11, s12, s13;  
42    wire c11, c12, c13, c14;  
43  
44    wire s21, s22, s23;  
45    wire c21, c22, c23, c24;  
46  
47    wire c31, c32, c33, c34;  
48  
49    assign out0 = pp00;  
50  
51    full_adder fa1 (.A(pp01), .B(pp10), .Cin(gnd), .out(out1), .cout(c11));  
52    full_adder fa11 (.A(pp02), .B(pp11), .Cin(c11), .out(s11), .cout(c12));  
53    full_adder fa12 (.A(ppn03), .B(pp12), .Cin(c12), .out(s12), .cout(c13));  
54    full_adder fa13 (.A(ppn13), .B(1), .Cin(c13), .out(s13), .cout(c14));  
55  
56    full_adder fa2 (.A(s11), .B(pp20), .Cin(gnd), .out(out2), .cout(c21));  
57    full_adder fa21 (.A(s12), .B(pp21), .Cin(c21), .out(s21), .cout(c22));  
58    full_adder fa22 (.A(s13), .B(pp22), .Cin(c22), .out(s22), .cout(c23));  
59    full_adder fa23 (.A(c14), .B(ppn23), .Cin(c23), .out(s23), .cout(c24));  
60  
61    full_adder fa3 (.A(s21), .B(ppn30), .Cin(gnd), .out(out3), .cout(c31));  
62    full_adder fa31 (.A(s22), .B(ppn31), .Cin(c31), .out(out4), .cout(c32));  
63    full_adder fa32 (.A(s23), .B(ppn32), .Cin(c32), .out(out5), .cout(c33));  
64    full_adder fa33 (.A(c24), .B(pp33), .Cin(c33), .out(out6), .cout(c34));  
65  
66    full_adder fa34 (.A(c34), .B(1), .Cin(0), .out(out7), .cout());  
67  
68 endmodule
```

1.2.6 block1_mod

```
1 module block1_mod (  
2     input A,B,Cin,N,  
3     output D,Cout,R  
4 );  
5     full_adder X1 (  
6         .A(A),  
7         .B(B),  
8         .Cin(Cin),  
9         .out(D),  
10        .cout(Cout)  
11    );  
12  
13    mux21 X2 (  
14        .A(D),  
15        .B(A),  
16        .S(N),  
17        .out(R)  
18    );  
19  
20 endmodule  
21
```

1.2.7 sign_mux

```
1 module sign_mux (  
2     input [3:0] A,  
3     input Sel,  
4     output [3:0] out  
5 );  
6     wire w0,w1,w2,w3,w4;  
7     assign w4 = Sel&1;  
8     AddSub x1(  
9         .A0(~A[0]), .A1(~A[1]), .A2(~A[2]), .A3(~A[3]), .B0(1), .B1(0), .B2(0), .B3(0), .Cin(0),  
10        .out0(w0), .out1(w1), .out2(w2), .out3(w3)  
11    );  
12    mux21 x6(  
13        .A(A[0]), .B(w0), .S(w4), .out(out[0])  
14    );  
15    mux21 x5(  
16        .A(A[1]), .B(w1), .S(w4), .out(out[1])  
17    );  
18    mux21 x4(  
19        .A(A[2]), .B(w2), .S(w4), .out(out[2])  
20    );  
21    mux21 x2(  
22        .A(A[3]), .B(w3), .S(w4), .out(out[3])  
23    );  
24 endmodule
```


1.2.8 Reminder_b

```
1  module Reminder_b (
2      input [3:0] A,
3      input [3:0] B,
4      output [3:0] out
5  );
6      wire d1,d2,d3,d4;
7      wire [2:0] cc1,cc2,cc3,cc4;
8      wire [2:0] ra1,ra2,ra3;
9
10     block1_mod x4(.A(0), .B(~B[3]), .Cin(cc1[0]), .D(d1),.N(d1), .Cout(), .R());
11     block1_mod x3(.A(0), .B(~B[2]), .Cin(cc1[1]), .D(),.N(d1), .Cout(cc1[0]), .R(ra1[0]));
12     block1_mod x2(.A(0), .B(~B[1]), .Cin(cc1[2]), .D(),.N(d1), .Cout(cc1[1]), .R(ra1[1]));
13     block1_mod x1(.A(A[3]), .B(~B[0]), .Cin(1), .D(),.N(d1), .Cout(cc1[2]), .R(ra1[2]));
14
15     block1_mod x12(.A(ra1[0]), .B(~B[3]), .Cin(cc2[0]), .D(d2),.N(d2), .Cout(), .R());
16     block1_mod x11(.A(ra1[1]), .B(~B[2]), .Cin(cc2[1]), .D(),.N(d2), .Cout(cc2[0]), .R(ra2[0]));
17     block1_mod x10(.A(ra1[2]), .B(~B[1]), .Cin(cc2[2]), .D(),.N(d2), .Cout(cc2[1]), .R(ra2[1]));
18     block1_mod x9(.A(A[2]), .B(~B[0]), .Cin(1), .D(),.N(d2), .Cout(cc2[2]), .R(ra2[2]));
19
20     block1_mod x16(.A(ra2[0]), .B(~B[3]), .Cin(cc3[0]), .D(d3),.N(d3), .Cout(), .R());
21     block1_mod x15(.A(ra2[1]), .B(~B[2]), .Cin(cc3[1]), .D(),.N(d3), .Cout(cc3[0]), .R(ra3[0]));
22     block1_mod x14(.A(ra2[2]), .B(~B[1]), .Cin(cc3[2]), .D(),.N(d3), .Cout(cc3[1]), .R(ra3[1]));
23     block1_mod x13(.A(A[1]), .B(~B[0]), .Cin(1), .D(),.N(d3), .Cout(cc3[2]), .R(ra3[2]));
24
25     block1_mod x20(.A(ra3[0]), .B(~B[3]), .Cin(cc4[0]), .D(d4),.N(d4), .Cout(), .R(out[3]));
26     block1_mod x19(.A(ra3[1]), .B(~B[2]), .Cin(cc4[1]), .D(),.N(d4), .Cout(cc4[0]), .R(out[2]));
27     block1_mod x18(.A(ra3[2]), .B(~B[1]), .Cin(cc4[2]), .D(),.N(d4), .Cout(cc4[1]), .R(out[1]));
28     block1_mod x17(.A(A[0]), .B(~B[0]), .Cin(1), .D(),.N(d4), .Cout(cc4[2]), .R(out[0]));
29
30 endmodule
31
```

1.2.9 Modulus (operation 3)

```
1 module Modulus(  
2     input [3:0] A,B,  
3     output [7:0] out ,  
4     output D_by_zero_f  
5 );  
6     wire a,b,c,d;  
7     assign a=B[0]|B[1];  
8     assign b=B[2]|B[3];  
9     assign c=a|b;  
10    assign d=~c;  
11    assign D_by_zero_f = d;  
12    wire [7:0] w;  
13  
14    sign_mux x2(  
15        .A(A),  
16        .Sel(A[3]),  
17        .out(w[3:0])  
18    );  
19    sign_mux x3(  
20        .A(B),  
21        .Sel(B[3]),  
22        .out(w[7:4])  
23    );  
24    wire [3:0] ww;  
25    Reminder_b x1(  
26        .A(w[3:0]),  
27        .B(w[7:4]),  
28        .out(ww[3:0])  
29    );  
30    wire [3:0] www;  
31    sign_mux x4(  
32        .A(ww[3:0]),  
33        .Sel(A[3]),  
34        .out(www[3:0])  
35    );  
36    wire [3:0] wwww;  
37  
38    mux21 x5(  
39        .A(wwww[0]),  
40        .B(0),  
41        .S(d),  
42        .out(wwwww[0])  
43    );  
44    mux21 x6(  
45        .A(wwww[1]),  
46        .B(0),  
47        .S(d),  
48        .out(wwwww[1])  
49    );  
50    mux21 x7(  
51        .A(wwww[2]),  
52        .B(0),  
53        .S(d),  
54        .out(wwwww[2])  
55    );  
56    mux21 x8(  
57        .A(wwww[3]),  
58        .B(0),  
59        .S(d),  
60        .out(wwwww[3])  
61    );  
62    AddSub x13(  
63        .A0(wwwww[0]),  
64        .A1(wwwww[1]),  
65        .A2(wwwww[2]),  
66        .A3(wwwww[3]),  
67        .B0(0),  
68        .B1(0),  
69        .B2(0),  
70        .B3(0),  
71        .Cin(0),  
72        .out0(out[0]),  
73        .out1(out[1]),  
74        .out2(out[2]),  
75        .out3(out[3]),  
76        .out4(out[4]),  
77        .out5(out[5]),  
78        .out6(out[6]),  
79        .out7(out[7])  
80    );  
81  
82 endmodule
```

1.2.10 selectors

```
1 module selectors (  
2     input wire [3:0] B,  
3     output wire [1:0] S  
4 );  
5  
6 wire ored_output;  
7  
8 assign ored_output = B[3]|B[2];  
9  
10 assign S[0] = ored_output|B[0];  
11 assign S[1] = ored_output |B[1];  
12  
13 endmodule
```

1.2.11 shifting_circuit

```
1 module shifting_circuit (  
2     input wire [3:0] A,  
3     input wire [1:0] S,  
4     output wire [7:0] Y  
5 );  
6  
7 assign Y[4] = Y[3];  
8 assign Y[5] = Y[4];  
9 assign Y[6] = Y[5];  
10 assign Y[7] = Y[6];  
11  
12 mux41 mux_instance3 (  
13     .A0(A[3]),  
14     .A1(A[3]),  
15     .A2(A[3]),  
16     .A3(A[3]),  
17     .S0(S[0]),  
18     .S1(S[1]),  
19     .out(Y[3])  
20 );  
21  
22 mux41 mux_instance2 (  
23     .A0(A[2]),  
24     .A1(A[3]),  
25     .A2(A[3]),  
26     .A3(A[3]),  
27     .S0(S[0]),  
28     .S1(S[1]),  
29     .out(Y[2])  
30 );  
31  
32 mux41 mux_instance1 (  
33     .A0(A[1]),  
34     .A1(A[2]),  
35     .A2(A[3]),  
36     .A3(A[3]),  
37     .S0(S[0]),  
38     .S1(S[1]),  
39     .out(Y[1])  
40 );  
41  
42 mux41 mux_instance0 (  
43     .A0(A[0]),  
44     .A1(A[1]),  
45     .A2(A[2]),  
46     .A3(A[3]),  
47     .S0(S[0]),  
48     .S1(S[1]),  
49     .out(Y[0])  
50 );  
51  
52 endmodule
```

1.2.12 ASR (operation 4)



```
1 module ASR(  
2     input wire [3:0] A,  
3     input wire [3:0] B,  
4     output wire [7:0] Y  
5 );  
6 wire [1:0] S;  
7 selectors selector_instance (  
8     .B(B),  
9     .S(S)  
10 );  
11 shifting_circuit shifting_instance (  
12     .A(A),  
13     .S(S),  
14     .Y(Y)  
15 );  
16  
17 endmodule
```

1.2.13 ALU_top (Top module for ALU)

```
1 module ALU_top(
2     input [3:0] A, B,
3     input [2:0] Sel,
4     output Sign_flag, DZ_Flag, Zero_Flag,
5     output [7:0] out
6 );
7     wire w1, w2, w3, w4, w5, cin, DZ_Flag_w, S0, S1, ws1, ws2, ws3, ws4, ws5, ws6, ws7;
8     wire [7:0] out_as, out_mul, out_mod, out_asr;
9
10    assign w1 = ~Sel[1];
11    assign cin = w1 & Sel[0];
12
13    AddSub inst0 (
14        .A0(A[0]), .A1(A[1]), .A2(A[2]), .A3(A[3]), .B0(B[0]), .B1(B[1]), .B2(B[2]), .B3(B[3]), .Cin(cin),
15        .out0(out_as[0]), .out1(out_as[1]), .out2(out_as[2]), .out3(out_as[3]), .out4(out_as[4]), .out5(out_as[5]),
16        .out6(out_as[6]), .out7(out_as[7])
17    );
18    multiplier inst1 (
19        .A0(A[0]), .A1(A[1]), .A2(A[2]), .A3(A[3]), .B0(B[0]), .B1(B[1]), .B2(B[2]), .B3(B[3]),
20        .out0(out_mul[0]), .out1(out_mul[1]), .out2(out_mul[2]), .out3(out_mul[3]), .out4(out_mul[4]), .out5(out_mul[5]),
21        .out6(out_mul[6]), .out7(out_mul[7])
22    );
23    Modulus inst2 (
24        .A(A), .B(B), .D_by_zero_f(DZ_Flag_w),
25        .out(out_mod)
26    );
27    ASR inst3 (
28        .A(A), .B(B),
29        .Y(out_asr)
30    );
31
32    assign w3 = ~Sel[0];
33    assign w4 = w3 & Sel[1];
34    assign S0 = w4 | Sel[2];
35    assign w5 = Sel[0] & Sel[1];
36    assign S1 = Sel[2] | w5;
37
38    mux41 out0_inst (.A0(out_as[0]), .A1(out_mul[0]), .A2(out_mod[0]), .A3(out_asr[0]), .S0(S0), .S1(S1), .out(out[0]));
39    mux41 out1_inst (.A0(out_as[1]), .A1(out_mul[1]), .A2(out_mod[1]), .A3(out_asr[1]), .S0(S0), .S1(S1), .out(out[1]));
40    mux41 out2_inst (.A0(out_as[2]), .A1(out_mul[2]), .A2(out_mod[2]), .A3(out_asr[2]), .S0(S0), .S1(S1), .out(out[2]));
41    mux41 out3_inst (.A0(out_as[3]), .A1(out_mul[3]), .A2(out_mod[3]), .A3(out_asr[3]), .S0(S0), .S1(S1), .out(out[3]));
42    mux41 out4_inst (.A0(out_as[4]), .A1(out_mul[4]), .A2(out_mod[4]), .A3(out_asr[4]), .S0(S0), .S1(S1), .out(out[4]));
43    mux41 out5_inst (.A0(out_as[5]), .A1(out_mul[5]), .A2(out_mod[5]), .A3(out_asr[5]), .S0(S0), .S1(S1), .out(out[5]));
44    mux41 out6_inst (.A0(out_as[6]), .A1(out_mul[6]), .A2(out_mod[6]), .A3(out_asr[6]), .S0(S0), .S1(S1), .out(out[6]));
45    mux41 out7_inst (.A0(out_as[7]), .A1(out_mul[7]), .A2(out_mod[7]), .A3(out_asr[7]), .S0(S0), .S1(S1), .out(out[7]));
46
47    assign w2 = Sel[0] & Sel[1];
48    mux21 inst4 (.A(0), .B(DZ_Flag_w), .S(w2), .out(DZ_Flag));
49
50    assign Sign_flag = out[7];
51
52    assign ws1 = out[0] | out[1];
53    assign ws2 = out[2] | out[3];
54    assign ws3 = out[4] | out[5];
55    assign ws4 = out[6] | out[7];
56    assign ws5 = ws1 | ws2;
57    assign ws6 = ws3 | ws4;
58    assign ws7 = ws5 | ws6;
59    assign Zero_Flag = ~ws7;
60
61 endmodule
```

2. ModelSim Waveform Verification

2.1 Simulation Setup

The ALU was verified using ModelSim with a dedicated testbench (ALU_tb). The simulation procedure was as follows:

1. All design files listed in src_files.list were compiled.
2. The testbench instantiated both ALU_top (DUT) and ALU_Gm (golden model as reference).
3. Input signals A and B were driven using binary counters at different frequencies to cover all possible combinations - 128 test case for every Sel.
4. The control signal Sel cycled through values 0 to 4.
5. Outputs were compared against the golden model every 1 ns.
6. Any mismatch was immediately reported.

A total of 640 test cases were executed. No mismatches were detected during simulation.

2.2 Files used

2.2.1 Golden Model

```
1  module ALU_Gm(  
2      input  signed [3:0] A, B,  
3      input  [2:0] Sel,  
4  
5      output signed [7:0] C,  
6      output Zero_F,  
7      output D_Zero_F,  
8      output Sign_F  
9  );  
10  
11      reg signed [7:0] C_temp;  
12  
13      always @(*) begin  
14          case (Sel)  
15              0: C_temp = A + B;  
16              1: C_temp = A - B;  
17              2: C_temp = A * B;  
18              3: C_temp = (B == 0) ? 8'sd0 : A % B;  
19              4: C_temp = A >>> B;  
20              default: C_temp = 0;  
21          endcase  
22      end  
23  
24      assign C          = C_temp;  
25      assign Zero_F     = (C_temp == 0);  
26      assign D_Zero_F  = (Sel == 3 && B == 0);  
27      assign Sign_F    = C_temp[7];  
28  
29  endmodule  
30
```

2.2.2 Testbench

```
1 module ALU_tb;
2     reg [3:0] A_tb, B_tb;
3     reg [2:0] Sel_tb;
4     wire [7:0] C_tb;
5     wire Zero_F_tb, D_Zero_F_tb, Sign_F_tb;
6     wire [7:0] C_exp;
7     wire Zero_F_exp, D_Zero_F_exp, Sign_F_exp;
8     integer error_count;
9
10    ALU_Gm GM (
11        .A(A_tb), .B(B_tb), .Sel(Sel_tb), .Zero_F(Zero_F_exp), .D_Zero_F(D_Zero_F_exp), .Sign_F(Sign_F_exp), .C(C_exp)
12    );
13
14    ALU_top DUT (
15        .A(A_tb), .B(B_tb), .Sel(Sel_tb), .Zero_Flag(Zero_F_tb), .DZ_Flag(D_Zero_F_tb), .Sign_flag(Sign_F_tb), .out(C_tb)
16    );
17
18    initial begin
19        forever begin
20            #128 A_tb[0] = ~A_tb[0];
21        end
22    end
23    initial begin
24        forever begin
25            #64 A_tb[1] = ~A_tb[1];
26        end
27    end
28    initial begin
29        forever begin
30            #32 A_tb[2] = ~A_tb[2];
31        end
32    end
33    initial begin
34        forever begin
35            #16 A_tb[3] = ~A_tb[3];
36        end
37    end
38    initial begin
39        forever begin
40            #8 B_tb[0] = ~B_tb[0];
41        end
42    end
43    initial begin
44        forever begin
45            #4 B_tb[1] = ~B_tb[1];
46        end
47    end
48    initial begin
49        forever begin
50            #2 B_tb[2] = ~B_tb[2];
51        end
52    end
53    initial begin
54        forever begin
55            #1 B_tb[3] = ~B_tb[3];
56        end
57    end
58    initial begin
59        error_count = 0;
60
61        A_tb = 0;
62        B_tb = 0;
63        Sel_tb = 0;
64        #128;
65        A_tb = 0;
66        B_tb = 0;
67        Sel_tb = 1;
68        #128;
69        A_tb = 0;
70        B_tb = 0;
71        Sel_tb = 2;
72        #128;
73        A_tb = 0;
74        B_tb = 0;
75        Sel_tb = 3;
76        #128;
77        A_tb = 0;
78        B_tb = 0;
79        Sel_tb = 4;
80        #128;
81
82        $display("Test done with %d errors out of 640 test case", error_count);
83        $finish;
84    end
```

```
1    initial begin
2        forever begin
3            #1;
4            if(C_exp != C_tb || Sign_F_exp != Sign_F_tb || D_Zero_F_exp != D_Zero_F_tb || Zero_F_exp != Zero_F_tb) begin
5                $display("Error: Input: A: %h, B: %h, Sel: %h, Output: out exp: %h found: %h, sign exp: %h found: %h, Dz exp: %h found: %h, Zero exp: %h found: %h", A_tb, B_tb, Sel_tb, C_exp, C_tb, Sign_F_exp, Sign_F_tb, D_Zero_F_exp, D_Zero_F_tb, Zero_F_exp, Zero_F_tb);
6                error_count = error_count + 1;
7            end
8        end
9    end
10
11
12
13 endmodule
```


2.2.3 Source list



```
1  mux21.v
2  mux41.v
3  full_adder.v
4  block1_mod.v
5  selector.v
6  sign_mux.v
7  shifting_circuit.v
8  Reminder_b.v
9  AddSub.v
10 Mult.v
11 Modulus.v
12 ASR.v
13 ALU_tb.v
14 ALU_Gm.v
15 ALU_top.v
```

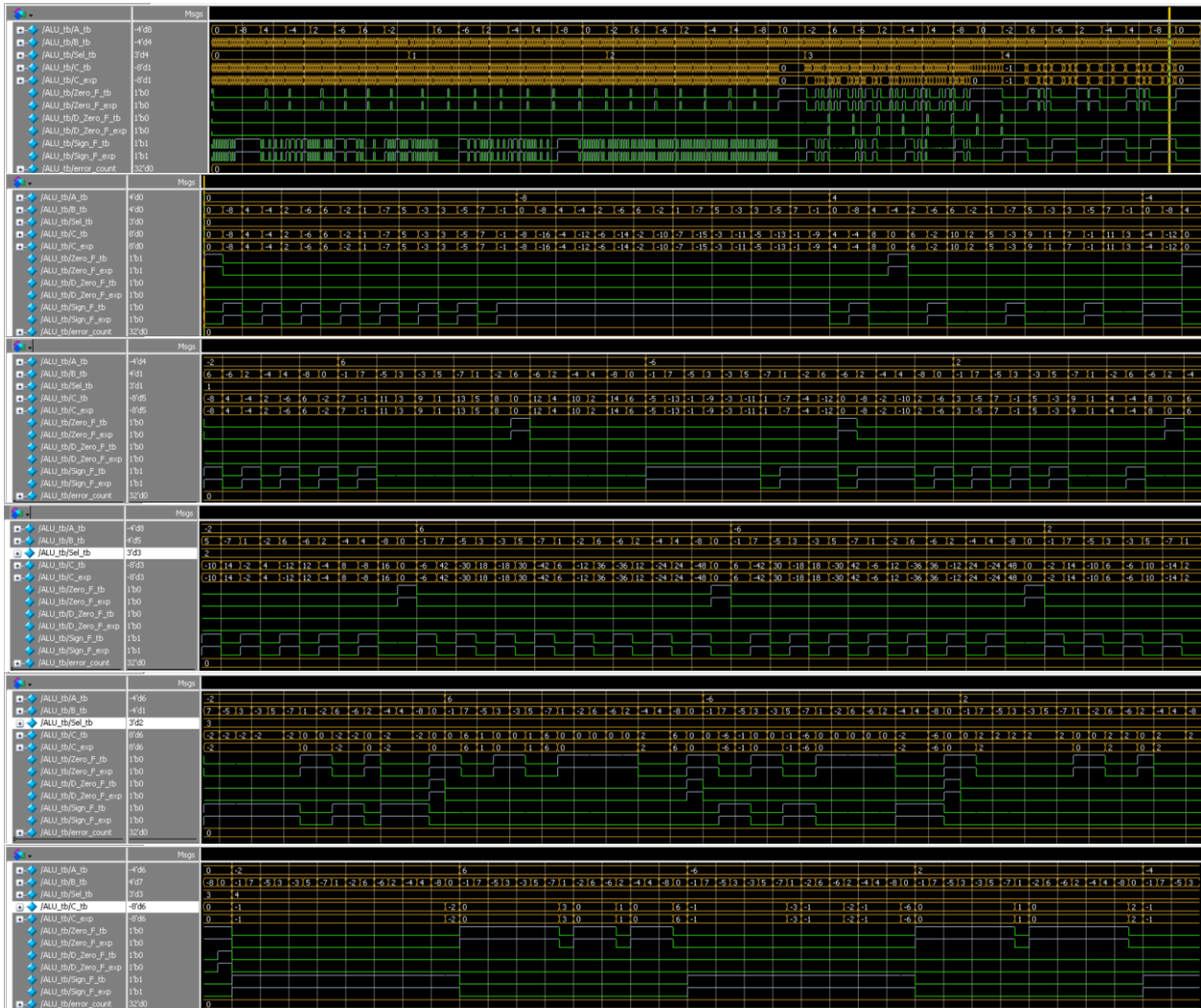
2.2.4 Do file



```
1  vlib work
2  vlog -f src_files.list
3  vsim -voptargs=+acc work.ALU_tb
4  add wave *
5  run -all
```

2.3 Simulation log and waveforms

```
# Test done with          0 errors out of 640 test case
# ** Note: $finish       : ALU_tb.v(83)
#   Time: 640 ns   Iteration: 0   Instance: /ALU_tb
# 1
# Break in Module ALU_tb at ALU_tb.v line 83
```



3. Conclusion

The designed ALU successfully meets all functional requirements using a fully structural, modular Verilog architecture. Comprehensive simulation and comparison against a behavioral golden model confirm the correctness of arithmetic operations and flag generation.

4. Appendix

Team Workload Distribution: Part B (Verilog & ModelSim)

Member Name	Assigned Work
Ahmed Shata	Design and implementation of the structural Verilog modules for addition and subtraction
Ahmed Ashraf	Design and implementation of the multiplication module using structural Verilog
Abdelrahman Elatroush	Design and implementation of the modulus operation using structural Verilog
Ahmed Nour	Development of the main ALU testbench , implementation of the golden behavioral model , design of all multiplexers and basic building blocks , and overall verification and debugging of the system
Moaz Khaled	Design and implementation of the arithmetic shift right (ASR) module using structural Verilog